

아이디어를 실제 서비스로 만든 과정

PeroChat — 웹 기반 AI 캐릭터 채팅 플랫폼

프로젝트 요약 학교 연구과제로 시작한 VRM × LLM 아이디어를 가입·결제·배포·운영이 가능한 웹 서비스로 확장했습니다. 기획부터 운영까지 혼자 진행한 프로젝트입니다.

[이미지 필요] 대표 화면 — 캐릭터 탐색과 2D·Live2D·VRM 채팅이 함께 보이는 장면

이름 / 지원 직무	안유찬 / Full-Stack Developer
프로젝트	PeroChat · 2025.05–현재 · 완전한 개인 프로젝트
현재 상태	실서비스 운영 · 실제 가입자 약 100 명 · 앱 배포 채널 테스트 중
연락처	[이메일 주소] [GitHub 주소] [서비스 주소]

기능을 만드는 것에서 끝내지 않고,
사용자가 들어오는 서비스가 되기까지 책임졌습니다.

01 · PROFILE

웹 서비스의 전체 수명주기를 경험한 개발자

게임·실시간 시스템 경험을 웹 제품 개발로 확장했습니다.

저는 Go 와 TypeScript 를 중심으로 프론트엔드, API 서버, 데이터베이스, AI 연동, 배포와 운영을 한 흐름으로 다룹니다. PeroChat 에서는 기획부터 출시 이후의 사용자 유입과 결제 검증까지 직접 수행했습니다.

게임 개발에서 익힌 상태 동기화, 비동기 처리, 3D 캐릭터 제어 경험은 브라우저의 VRM-Live2D 표현과 실시간 AI 응답을 설계하는 기반이 되었습니다. 지금은 기능 수보다 선택의 이유, 변경 비용, 장애 범위를 설명할 수 있는 구조를 더 중요하게 봅니다.

프로젝트가 시작된 배경

출발 청강문화산업대학교 전공심화 과정의 연구과제 — VRM 캐릭터와 LLM 을 결합하면 대화 결과가 표정과 애니메이션으로 이어지는 경험을 만들 수 있다고 보았습니다.

초기 구현 LLM 응답에 감정 태그를 포함 — 별도 분류 모델 없이 응답의 태그를 읽어 표정과 애니메이션을 선택했습니다.

확장 웹 기반 AI 캐릭터 채팅 서비스 — 2D-Live2D-VRM, 사용자 제작 콘텐츠, 여러 LLM 공급자, 결제와 운영 기능까지 제품 범위를 넓혔습니다.

[이미지 필요] 연구과제 초기 버전과 현재 PeroChat 화면 비교

제가 직접 맡은 범위

제품	문제 정의, 기능 기획, 사용자 흐름, 가격·결제 구조
프론트엔드	SvelteKit, TypeScript, 반응형 UI, PWA, Three.js-VRM, Live2D
백엔드	Go API, 인증·권한, 결제, 콘텐츠·채팅 도메인, 실시간 통신
데이터 / AI	PostgreSQL, Redis, LLM Gateway, 감정 분석 모델 연동
인프라 / 운영	Oracle Cloud, Coolify, Docker, GitHub Pages, 홍보와 사용자 대응

이름의 변화 처음에는 PersonaXi 였고, 줄여서 PXI 를 쓰다가 현재 PeroChat 으로 이름을 바꿨습니다. 과제용 프로토타입에서 실제 사용자가 들어오는 서비스로 바뀌면서 이름도 함께 정리했습니다.

02 · PRODUCT

PeroChat: 표현 방식을 사용자가 설계하는 AI 채팅

텍스트 응답을 넘어 캐릭터의 모습과 감정 표현까지 연결합니다.

사용자는 캐릭터를 만들고 공개하며, 2D 이미지·Live2D·VRM 중 원하는 표현 방식을 선택해 대화할 수 있습니다. 프롬프트, 로어북, 변수·상태창, 첫 장면과 인터랙티브 UI 를 조합해 각 캐릭터의 경험을 설계합니다.

현재 구현 범위

- 캐릭터 생성·편집·공개, 태그 검색, 피드, 세션과 저장 슬롯
- 2D·Live2D·VRM 채팅, 감정 기반 표정·모션, TTS·STT 와 음성 기능
- PromptKit, 로어북, 변수·상태 UI, 첫 장면, 사용자 제작형 인터랙션
- Supabase 인증·파일 저장·썸네일 CDN, 알림·관리자·외부 Agent API
- PayPal·PortOne·Google Play 결제 흐름과 웹훅·취소·중복 처리 검증
- 한국어·영어·일본어 UI, 약관·개인정보처리방침·라이선스 고지

[이미지 필요] 캐릭터 생성 화면 / 채팅 화면 / 제작 도구를 한 장에 조합

현재 운영 상태

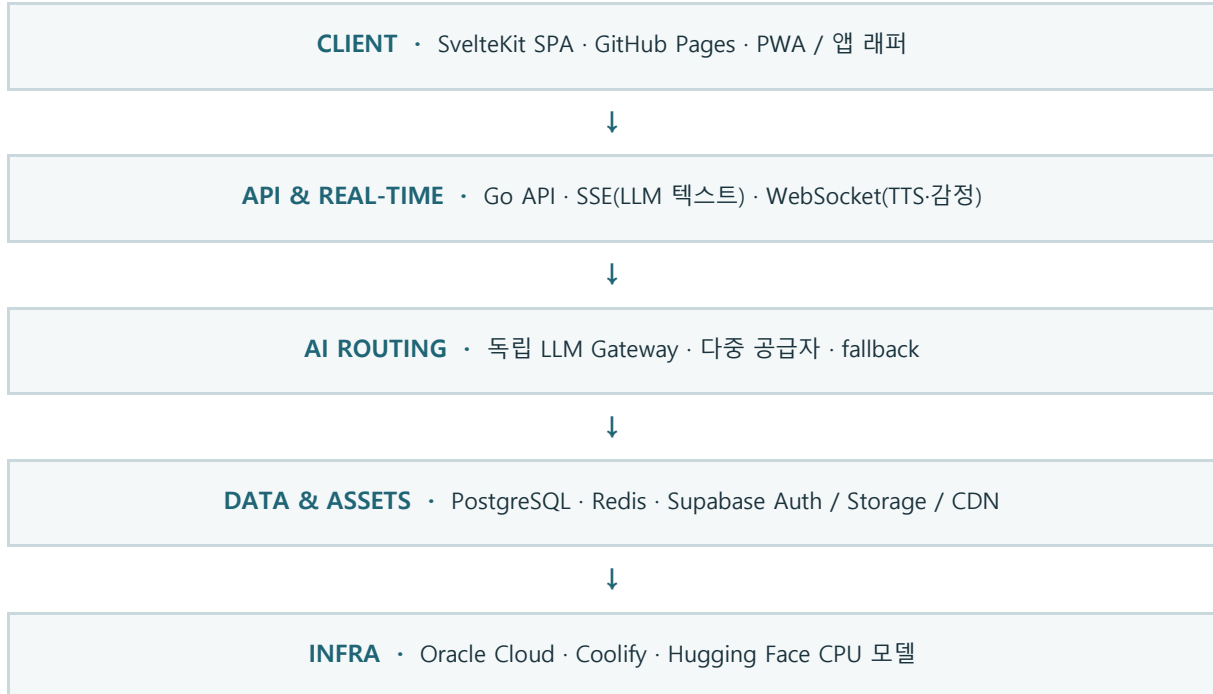
사용자	테스트 계정 없이 실제 가입자 약 100 명
유입	Twitter, Reddit, Instagram, YouTube Shorts 에서 직접 홍보
결제	실제 결제·취소·웹훅 검증 완료, 수익화는 초기 단계
앱	Google Play 및 Apps in Toss 테스트 단계
공개 범위	프론트엔드 공개 / 백엔드 요청 시 공개 가능

범위 명시 초기에 검토한 경매·방송 기능은 아직 개발하지 않았으며, 이 문서의 구현 기능과 성과에서 제외했습니다.

03 · ARCHITECTURE

정적 프론트엔드와 독립 API 를 중심으로 분리

비용, 앱 확장성, 공급자 교체 가능성을 함께 고려했습니다.



핵심 경계

프론트엔드	표현과 상호작용 담당. 정적 빌드로 배포 비용을 없애고 앱 패키징 경로를 단순화
Go API	인증·권한·결제·콘텐츠·채팅 등 제품의 비즈니스 규칙 담당
LLM Gateway	공급자별 요청 형식을 흡수하고 모델 라우팅·fallback 담당
모델 서버	Hugging Face Space 에서 CPU 감정 분석을 독립 실행
저장소	PostgreSQL 은 원본 데이터, Redis 는 반복 조회와 임시 상태, Supabase 는 인증·에셋 전달

통신 방식을 의도적으로 나눈 이유

LLM 텍스트 **SSE** — 서버에서 생성되는 토큰을 순서대로 전달하는 단방향 스트림에 적합합니다.

음성·감정 상태 **WebSocket** — 연결을 유지하며 재생·처리 상태를 양방향으로 주고받아야 합니다.

구조를 나눈 기준 자주 바뀌는 AI 공급자와 모델 서버를 제품 로직에서 분리했습니다. 공급자 하나가 막히거나 정책이 바뀌어도 서비스 전체를 다시 만들지 않기 위해서입니다.

04 · FRONTEND

Svelte 를 선택하고, SSR 보다 제품의 배포 경로를 택했습니다

친숙한 문법을 출발점으로 삼되 서비스 조건에 맞게 구조를 바꿨습니다.

왜 Svelte 였나

연구과제를 시작할 때 친구에게 '가장 빠르게 웹사이트를 만들 수 있는 선택'으로 추천받아 Svelte 를 사용했습니다. 이후 React 보다 Svelte 의 문법과 반응성 모델이 익숙해져, 제품을 확장하는 동안에도 일관되게 사용했습니다.

초기에는 SSR 을 고려했지만 로그인 이후의 상호작용이 중심인 서비스라 강한 이점을 찾기 어려웠습니다. 프론트엔드 호스팅 비용을 없애고, 백엔드를 독립 API 로 유지하며, 향후 Android·iOS 패키징을 단순화하기 위해 CSR 정적 빌드로 전환했습니다.

Framework	Svelte 5 · SvelteKit 2 · TypeScript · Vite
Rendering	CSR / adapter-static / GitHub Pages
UI	Tailwind CSS · 반응형 레이아웃 · 다국어
Character	Three.js · VRM · Live2D · 표정·모션 매핑
App	PWA · Capacitor 및 Apps in Toss 테스트

가장 오래 붙잡았던 문제: iPhone 키보드와 뷰포트

VRM 채팅에서 입력창을 누르면 키보드만 올라오길 원했지만, iPhone Safari/PWA 는 화면 전체를 밀어 캐릭터와 채팅 레이아웃이 무너졌습니다. 스크롤 컨테이너, 가시 영역과 입력 포커스의 동작을 며칠간 확인해 모바일 전용 레이아웃 보정을 적용했습니다.

[이미지 필요] iPhone 키보드 표시 전/후 또는 모바일 VRM 채팅 화면

배운 점 UI 버그는 CSS 한 줄의 문제가 아니라 브라우저가 뷰포트와 키보드를 해석하는 방식의 차이에서 생길 수 있었습니다. 실제 기기 재현과 상태 분리가 해결의 핵심이었습니다.

LLM 응답을 캐릭터의 표정과 움직임으로 연결

게임 개발 경험을 웹 기반 캐릭터 표현에 적용했습니다.

감정 표현 파이프라인의 진화

연구과제 응답 내 감정 태그 — LLM 이 반환한 태그로 표정과 애니메이션을 직접 선택했습니다.

현재 독립 감정 분석 + 자체 프리셋 — 기본 표정 외에도 GoEmotions 계열 분류에 대응하는 남·여 캐릭터 프리셋을 적용했습니다.

파라미터 탐색 런타임 로그와 직접 튜닝 — VRM 얼굴 파라미터 이름을 로그로 찾아낸 뒤 수치를 반복 조정해 표정을 구성했습니다.

[이미지 필요] 동일 대사에 대한 2D·Live2D·VRM 표현 비교

Live2D 도입에서 구현보다 먼저 확인한 것

Live2D 는 모델과 SDK 의 라이선스가 복잡해 약관과 일본어 문서를 조사하고 본사에 직접 문의했습니다.

상업적 이용이 허용된 모델을 사이트에서 표시하는 행위는 2 차 배포가 아니며, 원격으로 보여주는 형태에 가깝다는 답변을 받은 뒤 기능을 추가했습니다.

사용자 업로드 시 권리 확인과 고지 절차를 두었고, 혹시 모를 원본 추출 위험을 줄이기 위해 서버에 저장되는 Live2D 파일을 암호화했습니다. 모델별 파라미터 차이는 고급 편집 화면과 기본 제스처 매핑으로 흡수했습니다.

라이선스도 구현 범위로 기술적으로 된다고 바로 배포하지 않았습니다. 권리 범위를 먼저 확인하고 사용자 고지와 파일 보호까지 추가했습니다.

Go 로 제품 규칙을 모으고, Redis 로 반복 조회를 줄였습니다

캐릭터와 사용자가 늘어날 때 드러난 병목에서 출발했습니다.

왜 Go 였나

Go 문법과 고루틴에 익숙했고, 작은 언어 표면과 명시적인 오류 처리라는 철학을 선호했습니다. 표준 라이브러리와 웹 생태계가 단순하며, 동시 요청과 외부 API 호출을 다루기 쉽고 단일 바이너리 배포가 가능해 개인이 운영하는 API 서버에 잘 맞았습니다.

API	Go · REST · JWT/권한 미들웨어 · 외부 Agent API
Database	자체 Oracle 인스턴스의 PostgreSQL
Cache / State	동일 인스턴스의 Redis · 목록·필터 결과 캐시 · 토큰·임시 상태
Storage	Supabase Auth / Storage / CDN · 리사이즈 썸네일 전달
Reliability	graceful shutdown · 외부 서비스 상태 확인 · 작업 풀

Redis 를 넣은 이유

캐릭터가 늘수록 생성·편집 이후 목록을 다시 만들고, 사용자가 늘수록 태그 검색과 허브 조회가 PostgreSQL 에 반복적으로 집중됐습니다. '1,000 명이 동시에 캐릭터를 생성·편집하고, 다른 1,000 명이 조회'하는 시나리오를 시험했을 때 응답이 급격히 느려지는 병목을 확인했습니다.

캐릭터 목록·필터·허브성 데이터를 Redis 에 캐시하고, 변경 시 관련 키를 무효화하는 방식으로 반복 조회 압력을 줄였습니다. 원본 데이터는 PostgreSQL 에 유지해 캐시가 비어도 정상 동작하도록 설계했습니다.

[이미지 필요] 부하 테스트 화면 또는 캐시 적용 전·후 요청 흐름 다이어그램

정직한 한계 당시 체계적인 전후 지표를 보존하지 않아 개선 수치를 적지 않았습니다. 포트폴리오에서는 확인한 병목과 적용한 구조까지만 사실대로 제시합니다.

LLM 공급자를 기능이 아니라 교체 가능한 의존성으로 취급

요청 형식, 한도, 장애가 다른 공급자를 하나의 제품 흐름으로 묶었습니다.

독립 LLM Gateway

Gemini 를 포함한 여러 LLM 공급자는 모델명, 메시지 형식, 스트리밍 방식과 제한 정책이 다릅니다. 이를 Go API 의 비즈니스 로직에 직접 섞지 않고 별도 Gateway 에서 정규화해 공급자를 갈아끼우기 쉽게 만들었습니다.

사용자가 갑자기 늘면 무료·저가 모델의 rate limit 이나 quota 가 소진될 수 있습니다. 모델 라우팅과 fallback 을 Gateway 책임으로 두어 한 공급자의 실패가 곧바로 전체 채팅 중단으로 이어지지 않게 했습니다.

요청	공통 messages 형식 → 공급자별 payload 변환
응답	스트리밍 이벤트 정규화 → 프론트엔드 SSE 전달
장애	공급자 오류·한도 소진 시 다른 경로로 fallback
확장	비즈니스 로직 변경 없이 모델·공급자 추가 가능

감정 모델을 Hugging Face Space 로 분리

감정 분석 모델은 Docker 로 패키징해 Hugging Face Space 의 CPU 환경에서 실행합니다. GPU 가 필요하지 않은 작업에 별도 서버 비용을 쓰지 않고, API 와 독립적으로 모델을 교체할 수 있도록 했습니다. 서버에서는 상태 확인을 통해 모델 서비스의 가용성을 점검합니다.

[이미지 필요] LLM Gateway 라우팅과 감정 분석 병렬 흐름

실패를 전제로 좋은 모델 하나만 고르는 것보다 공급자가 막혔을 때 서비스가 계속 동작하는 편이 중요했습니다. 그래서 모델 선택과 fallback 을 Gateway 에 모았습니다.

저렴한 인프라 위에 반복 가능한 배포와 결제를 올렸습니다

혼자 운영할 수 있는 비용과 복잡도를 기준으로 선택했습니다.

인프라 선택

Frontend **GitHub Pages** — SSR의 이점이 크지 않아 정적 호스팅으로 프론트엔드 비용을 없앴습니다.

Compute **Oracle Cloud** — 개인 프로젝트가 감당할 수 있는 가격을 우선해 API·DB·Redis를 운영했습니다.

Deployment **Coolify** — Git 연동, 환경 변수, HTTPS, 로그와 서비스 관리를 한곳에 모으고 무중단 배포 체계를 직접 만드는 부담을 피했습니다.

Assets **Supabase CDN** — 인증과 파일 저장을 분리하고, 목록에서는 리사이즈 썸네일을 전달해 원본 전송을 줄였습니다.

[이미지 필요] Coolify 배포 화면 + Oracle 인스턴스 또는 서비스 구성

결제에서 확인한 실제 운영 경계

PayPal, PortOne, Google Play 결제를 연동하고 소액 실제 결제로 승인·취소·웹훅을 확인했습니다.

클라이언트의 성공 화면만 믿지 않고 서버 검증, 중복 처리 방지와 구매 상태 반영을 별도 흐름으로 다뤘습니다.

출시 이후에 한 일

- Twitter·Reddit·Instagram·YouTube Shorts 용 홍보 문구와 콘텐츠 제작
- 테스트 계정 없이 유입된 약 100 명의 실제 가입 흐름 확인
- 한국어·영어·일본어 UI 와 약관·개인정보처리방침·라이선스 문서 정비
- Google Play·Apps in Toss 테스트, 모바일 패키징과 결제 채널 검증

출시 후 알게 된 것 실제로 운영해 보니 배포보다 결제 실패, 외부 API 한도, 권리 고지, 모바일 입력 문제를 처리하는 데 더 많은 시간이 들었습니다.

09 · ENGINEERING FOUNDATION

게임 개발 기반과, 지금의 개발 방식

실시간 시스템 경험은 웹 제품의 상태·통신 문제를 이해하는 자산이 되었습니다.

게임 개발 경험 — 웹 직무와 연결되는 부분만

Unreal / C++	멀티플레이 애니메이션 위치 보정, 공유 인벤토리 상태와 선점 권한 동기화
Unity / C#	Vapor: 상태 패턴 기반 게임 로직, Google Sheets CSV 데이터 파이프라인, JSON 저장·복원
Networking	UDP ACK 기반 신뢰성 실험, NAT 홀펀칭, 입력 예측과 롤백 데모

이 경험을 통해 자연되는 네트워크, 여러 클라이언트가 공유하는 상태, 실패 후 복구 경로를 먼저 생각하는 습관을 얻었습니다. PeroChat 의 WebSocket 상태 처리와 캐릭터 런타임을 설계할 때도 같은 관점이 이어졌습니다.

AI 도구를 사용하는 방식

문서 해석, 초기 페이지 골격, 반복 작업과 버그 후보 탐색에 AI 에이전트와 자동화를 적극 사용했습니다. 다만 에이전트가 잘못된 가정으로 오래 헤매거나 실제 버그를 찾지 못하는 경우도 있었기 때문에, 아키텍처 결정·핵심 통합·재현·검증은 제가 직접 통제했습니다.

원칙 AI 가 작성한 코드의 양보다, 결과를 설명하고 실패했을 때 직접 고칠 수 있는지가 더 중요하다고 생각합니다.

다음 개선

- 부하 테스트와 캐시 적용 전·후 지표를 자동 수집해 의사결정 근거 보존
- 관측성, 통합 테스트, 결제 회귀 테스트를 강화해 운영 위험 축소
- 앱 테스트를 마무리하고 사용자 행동을 기반으로 수익화 가설 검증

[이미지 필요] 선택: 게임 프로젝트 네트워크 동기화 화면 1 장

안유찬 · Full-Stack Developer

[이메일 주소] · [GitHub 주소] · [서비스 주소]